

Practical No. 6

Aim: Implement standard queue operations and a circular queue using arrays.

Source-Code:

Standard Queue using Array

```
#include <iostream>
using namespace std;
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;
void enqueue(int value) {
    if (rear == SIZE - 1) {
        cout << "Queue is Full\n";
    } else {
        if (front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        cout << value << " inserted\n";
    }
}
void dequeue() {
    if (front == -1 || front > rear) {
        cout << "Queue is Empty\n";
    } else {
        cout << queue[front] << " deleted\n";
        front++;
    }
}
void display() {
    if (front == -1 || front > rear) {
        cout << "Queue is Empty\n";
    } else {
        cout << "Queue elements: ";
        for (int i = front; i <= rear; i++) {
            cout << queue[i] << " ";
        }
        cout << endl;
    }
}
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

Circular Queue using Array

```
#include <iostream>
using namespace std;
#define SIZE 5
int cqueue[SIZE];
int front = -1, rear = -1;
void enqueue(int value) {
    if ((rear + 1) % SIZE == front) {
        cout << "Circular Queue is Full\n";
    } else {
        if (front == -1)
            front = 0;
        rear = (rear + 1) % SIZE;
        cqueue[rear] = value;
        cout << value << " inserted\n";
    }
}
void dequeue() {
    if (front == -1) {
        cout << "Circular Queue is Empty\n";
    } else {
        cout << cqueue[front] << " deleted\n";
        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % SIZE;
        }
    }
}
void display() {
    if (front == -1) {
        cout << "Circular Queue is Empty\n";
    } else {
        cout << "Queue elements: ";
        int i = front;
        while (true) {
            cout << cqueue[i] << " ";
            if (i == rear)
                break;
            i = (i + 1) % SIZE;
        }
        cout << endl;
    }
}
int main() {
    enqueue(5);
    enqueue(10);
    enqueue(15);
    enqueue(20);
    display();
    dequeue();
    display();
    enqueue(25);
    display();
    return 0;
}
```

Output:

Standard Queue using Array

```
10 inserted
20 inserted
30 inserted
Queue elements: 10 20 30
10 deleted
Queue elements: 20 30
```

Circular Queue using Array

```
5 inserted
10 inserted
15 inserted
20 inserted
Queue elements: 5 10 15 20
5 deleted
Queue elements: 10 15 20
25 inserted
Queue elements: 10 15 20 25
```